

1. Unsupervised Learning Overview

머신러닝의 학습 방법은 크게 지도 학습(supervised learning)과 비지도 학습(unsupervised learning)으로 나눌 수 있었다.

- 지도 학습 : 라벨링이 된 데이터를 학습시키는 것
- 비지도 학습 : 라벨링이 되지 않은 데이터를 학습시키는 것

여기서 라벨링이란, train data 에 정답이 표시하는 작업을 지칭한다.

지금까진 입력-정답 쌍으로 구성된 데이터를 기반으로 지도 학습하는 방식의 머신러닝을 살펴보았으니 이번에는 레이블이 존재하지 않는, 즉 입력 값만 존재하는 데이터를 통해 학습하는 비지도 학습 방식에 대해 살펴보자.

2. 군집화 (Clustering)

비지도학습의 가장 대표적인 학습 방법으로는 군집화가 있다. 군집(clustering)은 비슷한 샘플 데이터끼리 하나의 그룹으로 모으는 작업을 의미하며, 군집 알고리즘을 통해 형성된 그룹을 클러스터(cluster)라고 부른다. 이 과정에서 데이터셋의 자연스러운 그룹화 또는 내재된 구조를 이해할 수 있다.

대표적인 알고리즘으로는 K-means, K-Medians, Mean-Shift, DBSCAN 등이 있는데, 우리는 그중 K-means, Mean-Shift, DBSCAN 에 대해 더 자세히 알아보자.

2-1. K-means

클러스터 중심까지의 거리를 기준으로 데이터를 k 개의 서로 다른 클러스터로 분할하는 방법이다.

알고리즘

1. training sample 에서 우리가 cluster 의 개수만큼 클러스터 중심(cluster center)과 센트로이드(centroid)를 임의로 지정한다. (Random 하게 K 개의 센트로이드를 초기화한다.)

$$\mu_1, \mu_2, \dots, \mu_k \in R^n$$

1. 모든 training sample 에 대해서 가장 가까운 센트로이드의 cluster 로 할당한다. (cluster assignment step)

$$\min_k \sum_i \|x^{(i)} - \mu_k\|^2$$

1. 각 cluster 에 해당하는 모든 점들의 평균을 구하고, 그 평균으로 센트로이드를 옮긴다(갱신한다). (move centroid step)

$$\mu_k = \frac{1}{m} (x^1, \dots, x^i, \dots, x^m)$$

1. 2 번과 3 번을 계속해서 반복하면, 센트로이드가 더 이상 움직이지 않고 수렴하는 지점에서 cluster 가 결정된다.

변수 설명:

- μ_k : cluster k 의 centriod 위치
- k : cluster 의 개수
- $\{x^i\}$: 각 cluster 에 있는 training set
- m : 각 cluster 에 있는 training set 의 개수

최적화

K-means 알고리즘을 최적화하는 방법은 다음과 같다. 아래는 K-means 알고리즘의 비용함수다. (Distortion function 이라고 부르기도 한다)

$$J(c^{(1)}, \dots, c^{(m)}, \mu^{(1)}, \dots, \mu^{(k)}) = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|^2$$

$$\min_{c, \mu} J(c^{(1)}, \dots, c^{(m)}, \mu^{(1)}, \dots, \mu^{(k)})$$

변수 설명:

- $c^{(i)}$: 각각의 training set(x^i)이 할당되어 있는 cluster 의 인덱스
- μ_k : cluster k 의 centriod 위치
- $\mu_{c^{(i)}}$: cluster 의 인덱스($c^{(i)}$)의 centriod 위치

알고리즘의 최종 목적은 비용함수 J 를 최소화하는 c 와 μ 를 찾는 것이다.

- 추가적으로 여러 번 랜덤 초기화(random initialization)을 통해 시행시켜 지역 최적점(local optima)에 빠지지 않고 전역적으로 가장 좋은 모델을 선택하는 것이 이상적이다.

최적 K 값 찾기

cluster 의 개수인 k 값은, 데이터를 직접 확인하거나 다른 clustering algorithm 의 결과를 보고 수동으로 결정할 수 있다.

그 중에서 Elbow Method 는 k 의 개수를 1 에서 점차 늘려가면서 비용함수 (Distortion function)을 계산한다. x 축을 k (cluster 의 개수), y 축을 비용함수로 설정하고 그래프를 그려보면, 특정 k 이후의 비용이 거의 변하지 않는 지점을 찾을 수 있다. 우리는 그곳을 elbow point 라고 일컫고, 해당 지점을 k 로 선택하는 것이 바람직하다.

실습 예제

이번 실습에서는 K-means 를 실제 데이터에 적용해보며 데이터가 어떻게 클러스터로 분리되는지를 확인해본다.

이를 위해 붓꽃(Iris) 데이터를 활용한다. 붓꽃 데이터는 꽃받침(sepal)과 꽃잎(petal)의 길이와 너비 정보를 포함하고 있으며, 세 가지 붓꽃 품종(Setosa, Versicolor, Virginica)으로 구성되어 있다.

실습을 통해 두 개의 feature (sepal length, sepal width)를 사용하여 데이터가 정말로 세 개의 군집으로 분리될 수 있는지 K-means 알고리즘을 통해 확인해보자!

우선 dataset 에서 DataFrame 을 활용해 구조를 파악하고, feature, target variable 을 설정한다.

※ 변수에 데이터가 어떻게 저장되어있는지 등 print 해보고 싶은 부분이 생긴다면 아래 코드에 print 문을 추가하여 꼭 확인해 보도록하자.

```
from sklearn.datasets import load_iris
iris = load_iris()
x_data = iris.data[:, :2] #use only 'sepal length and sepal width' #
'sepal length (cm)''sepal width (cm)'
y_data = iris.target

import pandas as pd
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target
df.head()

{"summary":{"\n  \"name\": \"df\",\n  \"rows\": 150,\n  \"fields\": [\n    {\n      \"column\": \"sepal length (cm)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.8280661279778629,\n        \"min\": 4.3,\n        \"max\": 7.9,\n        \"num_unique_values\": 35,\n        \"samples\": [\n          6.2,\n          4.5,\n          5.6\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"sepal width (cm)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.435866284936698,\n        \"min\": 2.0,\n        \"max\": 4.4,\n        \"num_unique_values\": 23,\n        \"samples\": [\n          2.3,\n          4.0,\n          3.5\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"petal length (cm)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1.7652982332594667,\n        \"min\": 1.0,\n        \"max\": 6.9,\n        \"num_unique_values\": 43,\n        \"samples\": [\n          6.7,\n          3.8,\n          3.7\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"petal width (cm)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.7622376689603465,\n        \"min\": 0.1,\n        \"max\": 2.5,\n        \"num_unique_values\": 22,\n        \"samples\": [\n          0.2,\n          1.2,\n          1.3\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"target\",\n      \"properties\": {
```

```
{\n      \"dtype\": \"number\", \n      \"std\": 0, \n      \"min\": 0, \n      \"max\": 2, \n      \"num_unique_values\": 3, \n      \"samples\": [\n        0, \n        1, \n        2\n      ], \n      \"semantic_type\": \"\", \n      \"description\": \"\" \n    } \n  ] \n}, \"type\": \"dataframe\", \"variable_name\": \"df\"}
```

target

1 50

```
Name: count, dtype: int64
```

```
km = KMeans(n_clusters=3, random_state=42)
```

```
KMeans(n_clusters=3, random_state=42)
```

```
array([2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2,
```

0, 2, 2, 2, 2, 2, 2, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0,

0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0,

0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0,

0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1],

```
dtype=int32)
```

```
np.unique(km.labels, return_counts=True)
```

`.transform` 메소드는 훈련 데이터 샘플에서 클러스터 중심까지 거리로 변환해준다.

```
array([[1.76483558, 1.05159358, 0.11840608],
       [1.91421501, 0.9261403 , 0.44093083],
       [2.11649197, 1.18751365, 0.38160189],
       [2.21291325, 1.24233499, 0.52193869],
       [1.88740674, 1.19250806, 0.17210462],
```

[1.6362795 , 1.26401576, 0.61483331],
[2.23658316, 1.37037386, 0.40696437],
[1.84176313, 1.04835901, 0.02863564],
[2.41906566, 1.38917649, 0.80375369],
[1.91293635, 0.96397369, 0.34470277],
[1.54505593, 1.07457758, 0.47876926],
[2.03892075, 1.20353254, 0.2078942],
[2.01414307, 1.02100579, 0.47499474],
[2.51386918, 1.50533642, 0.82560281],
[1.37197092, 1.30781396, 0.9785806],
[1.73068857, 1.70913197, 1.19432826],
[1.6362795 , 1.26401576, 0.61483331],
[1.76483558, 1.05159358, 0.11840608],
[1.32839928, 1.10998895, 0.78741349],
[1.86009779, 1.2962937 , 0.38369259],
[1.44978574, 0.80011792, 0.39499367],
[1.82341915, 1.21196862, 0.28778464],
[2.27431682, 1.48355768, 0.44093083],
[1.72755083, 0.90709988, 0.15880806],
[2.03892075, 1.20353254, 0.2078942],
[1.81429488, 0.83247755, 0.42804205],
[1.84176313, 1.04835901, 0.02863564],
[1.66796026, 0.99052111, 0.20692994],
[1.64529179, 0.91083623, 0.1960102],
[2.11649197, 1.18751365, 0.38160189],
[2.01292789, 1.05544411, 0.38732415],
[1.44978574, 0.80011792, 0.39499367],
[1.91121159, 1.51993049, 0.69944264],
[1.7292127 , 1.53217074, 0.91652605],
[1.91293635, 0.96397369, 0.34470277],
[1.81710723, 0.92522307, 0.22807893],
[1.38001155, 0.85263189, 0.49921939],
[1.9836475 , 1.2596795 , 0.2020396],
[2.41391488, 1.40759396, 0.74190296],
[1.74342716, 0.97690308, 0.0980816],
[1.86204131, 1.1182871 , 0.07224957],
[2.43899307, 1.33268066, 1.23629285],
[2.41602935, 1.46435639, 0.64747201],
[1.86204131, 1.1182871 , 0.07224957],
[1.86009779, 1.2962937 , 0.38369259],
[2.01414307, 1.02100579, 0.47499474],
[1.86009779, 1.2962937 , 0.38369259],
[2.21632386, 1.2786343 , 0.46563935],
[1.63699451, 1.11329869, 0.40052466],
[1.82674165, 0.98363976, 0.12814055],
[0.22542149, 1.32728976, 2.00699278],
[0.43143249, 0.80622577, 1.41252257],
[0.09089366, 1.19787548, 1.92219146],
[1.52419004, 0.47840143, 1.23143006],

[0.4161193 , 0.73433322, 1.62062334],
[1.14611553, 0.13031167, 0.9359594],
[0.56017281, 0.80388209, 1.30031535],
[2.02819644, 0.92123789, 1.03345053],
[0.27515171, 0.85207848, 1.67917242],
[1.65566916, 0.57363456, 0.7534056],
[2.10727361, 1.03823144, 1.4280126],
[0.91579866, 0.33251472, 0.99117102],
[1.19385214, 0.54200884, 1.57987974],
[0.73380816, 0.38681086, 1.214751],
[1.22525115, 0.27056893, 0.79474524],
[0.11562024, 1.01209665, 1.72546226],
[1.21505011, 0.35315291, 0.73213387],
[1.07977833, 0.02747211, 1.07722792],
[1.0677905 , 0.65141356, 1.71278136],
[1.34194443, 0.25917139, 1.10182576],
[0.92135767, 0.52305344, 0.92261585],
[0.76378534, 0.34367602, 1.26143569],
[0.7700276 , 0.5604917 , 1.59236302],
[0.76378534, 0.34367602, 1.26143569],
[0.4481237 , 0.65990279, 1.49064416],
[0.22542149, 0.88178635, 1.65046054],
[0.27476481, 1.03203408, 1.90074196],
[0.1351357 , 0.97613021, 1.7472321],
[0.83128071, 0.30714756, 1.12553099],
[1.2096975 , 0.11816202, 1.08037956],
[1.47589351, 0.40047142, 1.14053496],
[1.47589351, 0.40047142, 1.14053496],
[1.07977833, 0.02747211, 1.07722792],
[0.89488259, 0.22654085, 1.23207954],
[1.41472723, 0.48389146, 0.58173877],
[0.87553385, 0.74289083, 0.99439429],
[0.11562024, 1.01209665, 1.72546226],
[0.92883246, 0.65660648, 1.71663042],
[1.21505011, 0.35315291, 0.73213387],
[1.43295779, 0.33449483, 1.05129444],
[1.39587765, 0.28878405, 0.96416804],
[0.71664552, 0.44847751, 1.17474252],
[1.11839834, 0.09615239, 1.14717915],
[1.97127401, 0.86744039, 1.12801596],
[1.26926263, 0.1737489 , 0.93958501],
[1.11525494, 0.31622777, 0.81536495],
[1.12636015, 0.22020574, 0.87202064],
[0.63711948, 0.4742422 , 1.30553437],
[1.80653835, 0.70053888, 0.93274863],
[1.14611553, 0.13031167, 0.9359594],
[0.56017281, 0.80388209, 1.30031535],
[1.07977833, 0.02747211, 1.07722792],
[0.29673033, 1.36160283, 2.13729268],

```
[0.5416346 , 0.56585217, 1.39757647],
[0.321509 , 0.78883721, 1.55409781],
[0.79074834, 1.85212779, 2.62907208],
[1.99716979, 0.89453266, 0.93403426],
[0.51752886, 1.5404606 , 2.35397961],
[0.58543124, 0.94619396, 1.93153307],
[0.6527894 , 1.69065132, 2.2007317 ],
[0.33701752, 0.88616196, 1.51129746],
[0.55731686, 0.62646056, 1.57264745],
[0.07555439, 1.07150045, 1.84434812],
[1.25230254, 0.20604085, 1.1588011 ],
[1.04929863, 0.11074363, 1.01233394],
[0.43143249, 0.80622577, 1.41252257],
[0.321509 , 0.78883721, 1.55409781],
[1.14611553, 2.22210167, 2.71956246],
[1.0061333 , 1.92863232, 2.81837187],
[1.19385214, 0.54200884, 1.57987974],
[0.15286608, 1.23548173, 1.90767398],
[1.24343637, 0.20420116, 0.86441888],
[0.92871792, 1.92941481, 2.76622848],
[0.6349451 , 0.52646919, 1.48472893],
[0.25215235, 1.10786213, 1.69882901],
[0.40707305, 1.51402251, 2.20581504],
[0.67142747, 0.43976838, 1.34908117],
[0.71664552, 0.44847751, 1.17474252],
[0.49568989, 0.63558026, 1.52892773],
[0.39432943, 1.45919337, 2.23535679],
[0.64821027, 1.62996701, 2.47499899],
[1.30708623, 2.39756165, 2.91781082],
[0.49568989, 0.63558026, 1.52892773],
[0.58160266, 0.53728879, 1.43833932],
[0.85624487, 0.33925557, 1.37201312],
[0.89035372, 1.95081018, 2.72778665],
[0.60737135, 0.88189333, 1.2943029 ],
[0.41355485, 0.74732226, 1.43206843],
[0.81617032, 0.38190189, 1.08222918],
[0.09089366, 1.19787548, 1.92219146],
[0.11562024, 1.01209665, 1.72546226],
[0.09089366, 1.19787548, 1.92219146],
[1.07977833, 0.02747211, 1.07722792],
[0.12617936, 1.14504676, 1.80843026],
[0.25215235, 1.10786213, 1.69882901],
[0.1351357 , 0.97613021, 1.7472321 ],
[0.7700276 , 0.5604917 , 1.59236302],
[0.321509 , 0.78883721, 1.55409781],
[0.69386825, 0.82610703, 1.19432826],
[0.91579866, 0.33251472, 0.99117102]])
```

.predict 메소드는 가장 가까운 centroid 의 cluster 로 출력한다.

```

y_predict = km.predict(x_data)
y_predict
array([2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2,
      2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2,
      2, 2, 2, 2, 2, 2, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1,
0,
      1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0,
1,
      1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0,
0,
      0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0,
0,
      0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1]),
dtype=int32)

```

KMeans 는 반복적으로 클러스터 중심을 옮기면서 최적의 클러스터를 찾는데, `n_iter_` 메소드는 이 알고리즘이 반복한 횟수를 출력한다.

```

print(km.n_iter_)

8

import matplotlib.pyplot as plt

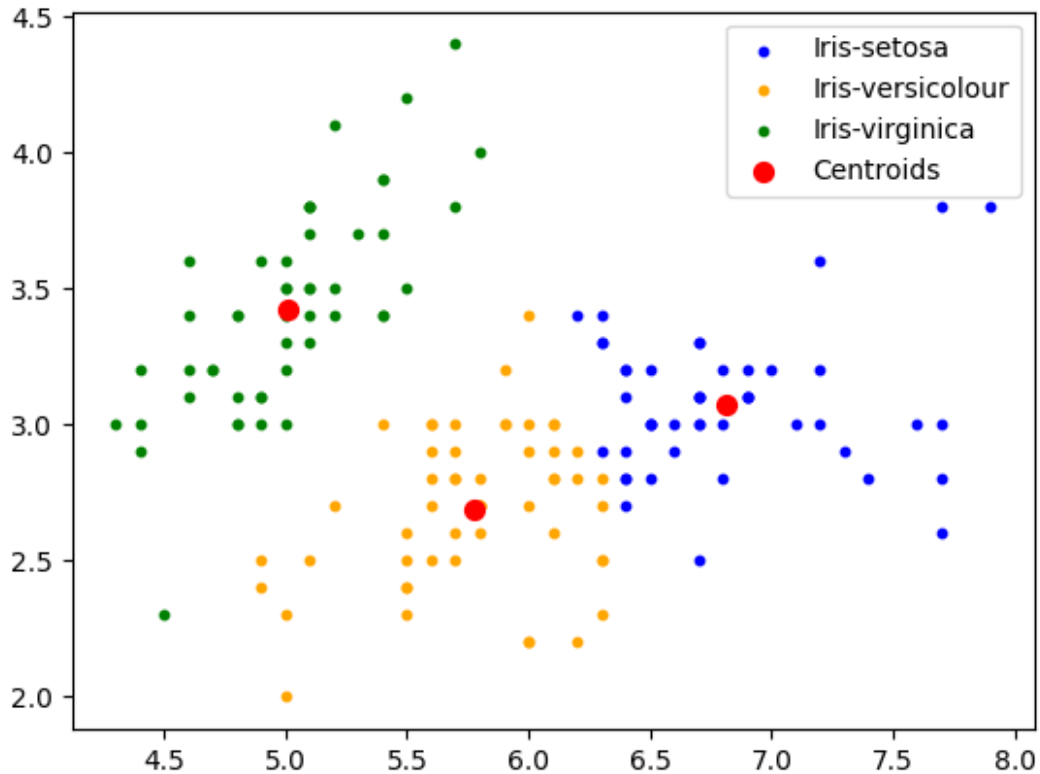
#Visualising the clusters
plt.scatter(x_data[y_predict == 0, 0], x_data[y_predict == 0, 1], s =
10, c = 'blue', label = 'Iris-setosa')
plt.scatter(x_data[y_predict == 1, 0], x_data[y_predict == 1, 1], s =
10, c = 'orange', label = 'Iris-versicolour')
plt.scatter(x_data[y_predict == 2, 0], x_data[y_predict == 2, 1], s =
10, c = 'green', label = 'Iris-virginica')

#Plotting the centroids of the clusters
plt.scatter(km.cluster_centers_[0, 0], km.cluster_centers_[0, 1], s =
50, c = 'red', label = 'Centroids')

plt.legend()

<matplotlib.legend.Legend at 0x79b1c0eea720>

```

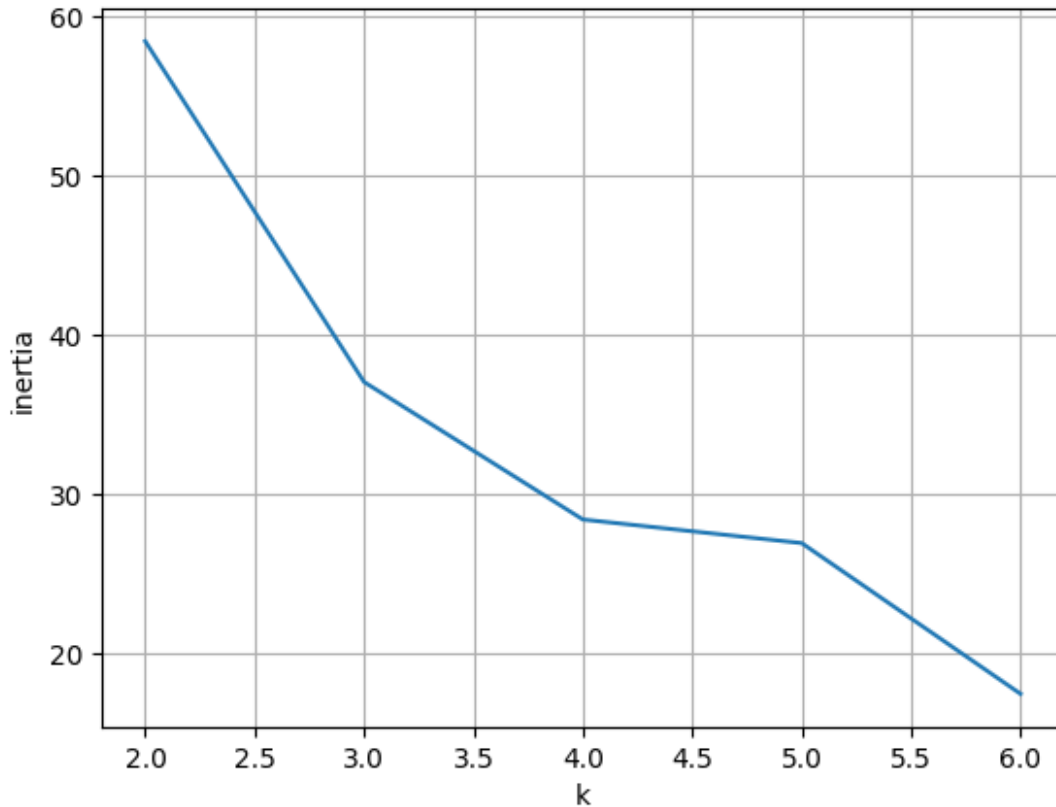



최적의 k 를 찾기 위해서, Elbow method 를 진행해보자.

centriod 와 샘플 사이의 거리의 제곱 합을 inertia 라고 부르는데, inertia 는 클러스터에 속한 샘플이 얼마나 가깝게 모여있는지를 나타낸다고 생각하면 된다. 일반적으로 클러스터 개수가 늘어나면 inertia 도 줄어든다. KMeans 에는 `inertia_` 메소드를 통해 Elbow method 를 실행할 수 있다.

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

inertia = []
for k in range(2, 7):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(x_data)
    inertia.append(km.inertia_)
plt.plot(range(2, 7), inertia)
plt.xlabel('k')
plt.ylabel('inertia')
plt.grid()
plt.show()
```



2-2. Mean-Shift Clustering

데이터셋 내 데이터 포인트들의 평균값으로 탐색 창을 반복적으로 이동시켜 밀집 영역(모드)을 식별하는 비모수적 밀도 기반 클러스터링 알고리즘을 말한다.

Mean-Shift Clustering 의 필요성

여태까지, clustering 이 무엇인지 이해하고 clustering 의 가장 대표적이고 간단한 예시인 K-means 에 대해 알아보았다. K-means 는 전체 N 개의 데이터를 K 개의 cluster 로 빠르게 묶어낸다는 장점을 가지고 있지만, 두 가지의 한계점을 가지고 있다.

- cluster 의 개수 K 가 사전에 결정되어야 한다.
- 초기 중심점(Centroid) 설정에 따라 수렴 여부가 크게 좌우된다.

이와는 대조적으로, 시간이 좀 더 걸리더라도, 데이터에 따라 적절한 클러스터 개수 K 를 발견하는 것부터 컴퓨터가 스스로 찾아내어 clustering 할 수 있다면 어떨까? 이러한 경우 비모수 기법(nonparametric method) 방식을 사용할 수 있다. 이때 평균 이동(Mean-shift) clustering 이 바로 대표적인 비모수 클러스터링 기법 알고리즘이다.

잠깐, 비모수 기법?

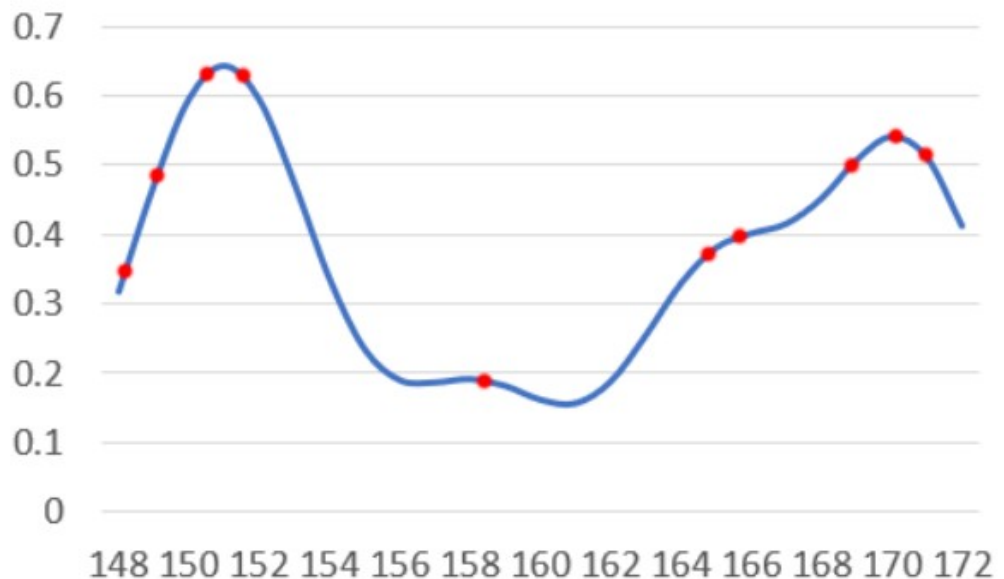
여기서 모수적(parametric) 방법과 비모수적(nonparametric) 방법의 차이를 이해할 필요가 있다.

- 모수: 데이터의 구조를 설명하기 위해 모델의 형태와 필요한 파라미터 수가 사전에 정해져 있는 방법. 즉, 알고리즘이 동작하기 전에 사용자가 모델의 구조나 하이퍼파라미터를 미리 설정해야 한다.
- 비모수: 모델의 구조나 필요한 파라미터 수가 미리 고정되어 있지 않고, 데이터의 분포나 구조에 따라 모델이 유연하게 결정되는 방법. 사용자가 클러스터 개수와 같은 핵심 파라미터를 직접 지정하지 않더라도 스스로 구조를 찾아낸다. 구분 특징 예시 모수적 방법 (Parametric) 모델 구조와 파라미터 수가 사전에 결정됨 K-means 비모수적 방법 (Nonparametric) 데이터 구조에 따라 모델 복잡도가 결정됨 Mean-shift, DBSCAN

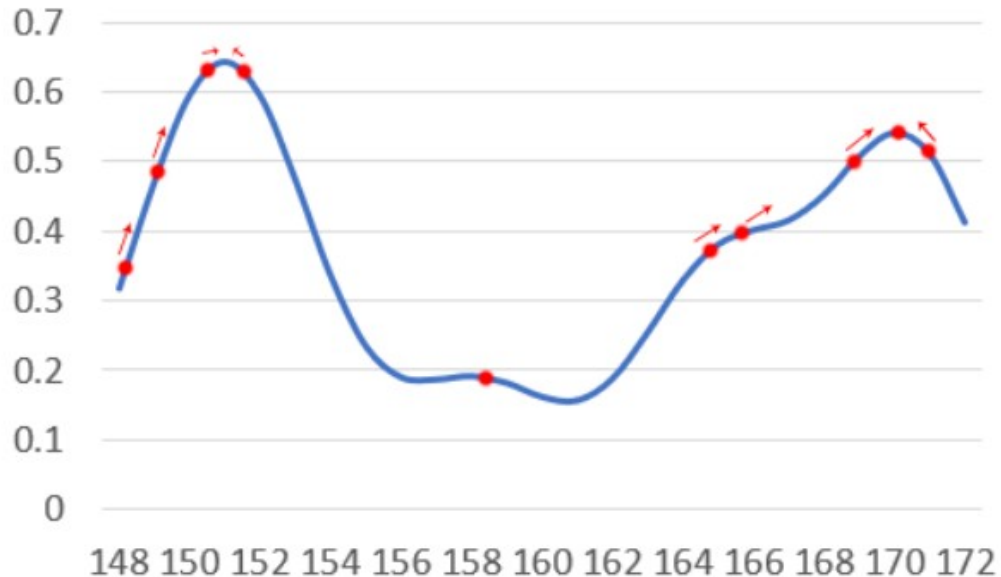
이제 K-means 에서 Mean-Shift 로 이어지는 흐름이 어느정도 이해됐을 것이다. 본격적으로 Mean-Shift Clustering 에 대해 공부해보자!

직관

1. 'Mean-Shift Clustering 의 필요성' 파트에서 Mean-Shift Clustering 은 클러스터의 개수 k 부터 스스로 찾는다고 했다. 각 데이터마다 밀도가 높은 가까운 부분으로 이동하다보면 자연스레 데이터들이 모이는 지점들이 생길 것이고, 적절한 k 가 보이기 시작할 것이다. 이를 토대로 클러스터를 형성할 수 있다. 그림과 함께 차근차근 살펴보자.
2. 데이터 x 들이 아래와 같은 그래프를 형성하고 있다. 그중에서도 빨간 점들이 우리가 관측하고 있는 데이터들이라고 하자.



3. 빨간 점 데이터들 각각에 대하여 주변의 밀도를 계산하고, 밀도가 높은 쪽으로 조금씩 이동시킨다면, 아래 그래프에 표시된 것처럼 움직일 모습을 상상해볼 수 있다.



4. 위 그래프 상에 나타난 방향대로 점들이 움직이다보면, 왼쪽-중간-오른쪽 이렇게 총 3개의 클러스터가 형성될 것이라는게 이해될 것이다! 이것이 바로 Mean-Shift clustering의 아이디어이다. (실제로는 위 그래프에서 본 것처럼 연속적인 분포가 주어지는 않고 점들만 주어지므로 이에 대한 계산 과정이 필요)

알고리즘

Mean-Shift 알고리즘은 다음과 같은 과정을 반복한다.

1. 각 데이터 포인트를 하나의 초기 중심점으로 설정한다.
2. 현재 중심점 주변에 반경 h (bandwidth)를 가지는 탐색 영역을 설정한다. 이 영역 안에 포함된 데이터들을 찾는다.
3. 탐색 영역 안에 있는 데이터들의 평균 위치를 계산한다. 계산된 평균 위치는 현재 위치보다 데이터 밀도가 높은 방향을 나타낸다.
4. 현재 중심점을 새로 계산된 평균 위치로 이동시킨다.
5. 중심점의 이동이 거의 없을 때까지 [2-4]단계를 반복한다.
6. 같은 위치로 수렴한 중심점들을 하나의 cluster로 묶는다. 일련의 과정을 통해 클러스터 개수가 자동으로 결정된다.

커널 밀도 추정, KDE

이러한 Mean-Shift는 데이터 밀도를 계산하기 위해 KDE(Kernel Density Estimation)라는 방법을 사용한다.

KDE의 아이디어는 다음과 같다, 각 데이터 포인트 주변에 작은 확률 분포(커널)를 놓고, 이를 모두 합쳐서 전체 데이터 분포의 밀도를 추정한다는 것.

밀도? 추정? 커널? 이런 용어들은 처음에는 낯설게 느껴질 수도 있지만 하나씩 살펴보면 생각보다 어렵지만은 않다.

- 밀도: 어떤 위치 x 에서, 그 주변에 데이터가 얼마나 많이 모여 있는지 나타내는 값. 지점 $x=a$ 에서의 함수값 $f(a)$ 로 표현할 수도 있다.
- 밀도 추정: 관측된 데이터(표본)의 분포를 통해, 원래 변수(모집단)의 확률 밀도(분포)를 추정하는 것. 현실에서는 데이터가 실제로 어떤 정답 확률 분포를 따르는지 정확히 파악할 수 없으므로, 이러한 추정 과정이 필요하다.
- 커널 함수: 각 데이터 포인트가 주변 공간에 얼마나 영향을 미치는지 나타내는 함수. 원점을 중심으로 대칭이면서 적분값이 1 인 non-negative 함수, 대체로 종 모양 형태 (ex: Gaussian, Uniform, etc) 실제 KDE에서는 Gaussian kernel 이 가장 많이 사용된다.
- 커널 밀도 추정: Non-Parametric Density Estimation 중의 한 방식으로 Kernel function 을 이용하여 Histogram 의 문제점을 개선한 밀도 추정 방식.

각 데이터 x_i 주변에 kernel 함수 K 를 놓고, 그것을 모두 합쳐서 밀도를 추정.

$$KDE = \frac{1}{n} \sum_{i=1}^n K_h(x - x_i) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

변수 설명:

- n : 데이터 개수
- x_i : i 번째 데이터
- K : kernel 함수
- h : bandwidth (커널의 폭)

Histogram 을 smoothing 했다고도 볼 수 있다. 히스토그램은 데이터의 확률 밀도를 추정하는 가장 간단한 방식이지만, discrete 한 경계에서 불연속성을 가진다.

수행 절차:

이제 KDE 개념과 함께 좀 더 엄밀한 mean-shift clustering 알고리즘을 다시 정리하자면,

- a. KDE 를 이용해 데이터 공간의 확률 밀도를 추정한다.
- b. 각 데이터 포인트를 초기 중심으로 두고, 주변 데이터의 밀도를 계산하여 밀도가 증가하는 방향(gradient 방향)으로 중심을 이동시킨다.
- c. 이 과정을 반복하면 중심점은 데이터 밀도가 가장 높은 지점(mode)에 수렴할 수 있다.
- d. 동일한 mode 로 수렴한 데이터 포인트들을 하나의 cluster 로 묶는다.
- e. 이때 mode 의 개수만큼 cluster 가 형성되므로, 별도로 군집 개수를 지정할 필요가 없다.

알고리즘의 장단점:

- 장점: K-Means 알고리즘과 대조적으로, 군집의 개수를 지정해 줄 필요가 없으며 평균이동 알고리즘이 알아서 군집의 개수를 알아낸다.
- 단점: 탐색 창의 크기, 즉 bandwidth 크기 설정이 필요하다.

실습 예제

이번에는 밀도 기반 클러스터링 알고리즘인 Mean-shift 가 데이터의 밀도 분포를 기반으로 어떻게 클러스터를 형성하는지 확인해보겠다.

실제 데이터 대신 `make_blobs` 함수를 이용하여 여러 개의 밀집된 데이터 군집을 가진 가상의 데이터를 생성하고, Mean-shift 알고리즘을 적용하여 데이터의 밀도 중심(mode)을 찾아 클러스터가 형성되는 과정을 살펴본다.

또한 bandwidth 값을 변경하면서 클러스터 개수가 어떻게 변하는지도 확인한다.

먼저 필요한 라이브러리들을 import 하자. 사이킷런은 Mean-Shift Clustering 을 위해 MeanShift 클래스를 제공한다.

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import MeanShift
```

`make_blobs` 함수를 사용하여 가상 데이터셋을 생성해보자.

X에는 feature 을, y에는 각 data point 의 군집 label 을 할당한다. MeanShift 클러스터링 알고리즘을 사용하여 데이터를 클러스터링한다.

```
X, y = make_blobs(n_samples=200, n_features=2, centers=3,
                  cluster_std=0.7, random_state=0)

meanshift = MeanShift(bandwidth=0.8)
cluster_labels = meanshift.fit_predict(X)
```

bandwidth 값을 다르게 설정하여 클러스터링을 수행해보자.

```
meanshift = MeanShift(bandwidth=1)
cluster_labels = meanshift.fit_predict(X)
print(np.unique(cluster_labels)) # 유형 출력해보기

[0 1 2]
```

`estimate_bandwidth` 함수를 통해 최적의 bandwidth 값을 추정해보자.

```
from sklearn.cluster import estimate_bandwidth

bandwidth = estimate_bandwidth(X)
print(round(bandwidth, 3)) # bandwidth 값
```

1.816

최적의 bandwidth 를 사용하여 다시 클러스터링을 수행해보자.

```
best_bandwidth = estimate_bandwidth(X)

meanshift = MeanShift(bandwidth=best_bandwidth)
cluster_labels = meanshift.fit_predict(X)
print(np.unique(cluster_labels)) # 유형 출력해보기

[0 1 2]
```

클러스터링 결과를 시각화해보자.

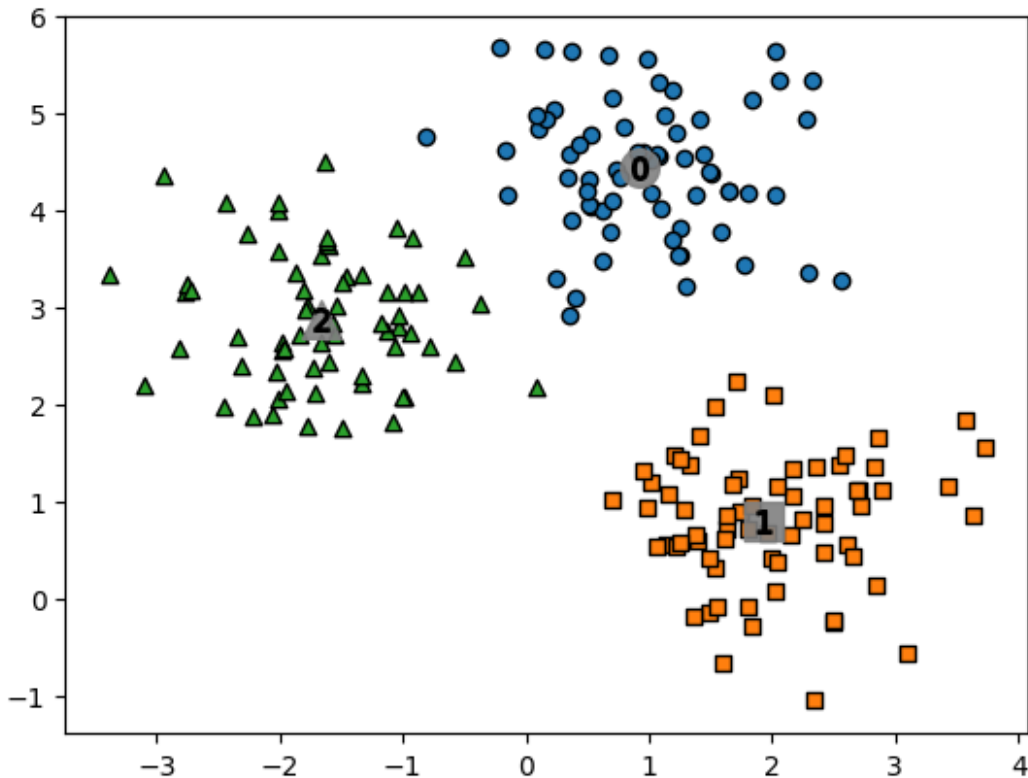
```
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

clusterDF = pd.DataFrame(data=X, columns=['ftr1', 'ftr2'])
clusterDF['target'] = y

clusterDF['meanshift_label'] = cluster_labels
centers = meanshift.cluster_centers_
unique_labels = np.unique(cluster_labels)
markers=['o', 's', '^', 'x', '*']

for label in unique_labels:
    label_cluster = clusterDF[clusterDF['meanshift_label'] == label]
    center_x_y = centers[label]
    plt.scatter(x=label_cluster['ftr1'], y=label_cluster['ftr2'],
edgecolor='k', marker=markers[label])
    plt.scatter(x=center_x_y[0], y=center_x_y[1], s=200, color='gray',
alpha=0.9, marker=markers[label])
    plt.scatter(x=center_x_y[0], y=center_x_y[1], s=70, color='k',
edgecolor='k', marker='$%d$' % label)

plt.show()
```



```
print(clusterDF.groupby('target')['meanshift_label'].value_counts())
```

target	meanshift_label	count
0	0	67
1	1	67
2	2	66

Name: count, dtype: int64

2-3. DBSCAN

DBSCAN 은 Density-Based Spatial Clustering of Applications with Noise 의 약자로, 데이터가 밀집되어 있는 영역을 하나의 군집으로 간주하는 클러스터링 알고리즘이다.

군집 알고리즘 분류

지금까지 살펴본 클러스터링 알고리즘을 조금 더 넓은 관점에서 정리해보면 다음과 같이 분류할 수 있다.

계층적(Hierarchical) 방법과 비계층적(Non-hierarchical) 방법.

이 중 비계층적 클러스터링은 데이터를 하나의 구조로 단계적으로 묶는 것이 아니라, 데이터의 특성을 기준으로 직접 클러스터를 형성하는 방법이다.

비계층적 클러스터링은 다시 클러스터를 형성하는 기준에 따라 다음과 같이 나눌 수 있는데,

- Distance-based : 데이터 간 거리(distance)를 기준으로 클러스터를 형성
- Density-based : 데이터가 얼마나 밀집되어 있는지(density)를 기준으로 클러스터를 형성

가장 처음 살펴봤던 K-means clustering 은 대표적인 distance-based 알고리즘이다. 반면, Mean-shift와 DBSCAN 은 density-based 알고리즘에 속한다. 특히 DBSCAN 은 밀도 낮은 데이터 포인트를 noise 로 분류할 수 있다는 점이 큰 특징이다. 이러한 특성 덕분에 복잡한 형태의 데이터 분포에서도 효과적으로 클러스터를 찾을 수 있다.

알고리즘

DBSCAN 의 기본 아이디어는 다음과 같다.

특정 점 주변에 충분한 개수의 이웃 데이터가 존재하면 그 점을 중심으로 클러스터를 확장하고, 이러한 과정을 반복하여 밀도가 높은 영역 전체를 하나의 클러스터로 형성한다.

주요 개념:

DBSCAN 은 두 개의 중요한 파라미터를 사용한다.

- Eps (ϵ) : 한 점 주변에서 이웃을 탐색하는 반경
- MinPts : 군집 생성에 필요한 최소 데이터 포인트 수 (자기 자신도 포함)
 - MinPts = 4 라면, 아래 이미지에서 Eps 내의 점이 5 개이므로 이들을 하나의 군집으로 판단할 수 있다

이 두 기준 값을 이용하여 데이터 포인트를 다음 세 가지 유형으로 분류한다.

- core point : 설정한 반경(Eps) 안에 MinPts 개수 이상의 데이터 포인트(자기 자신 포함)가 존재하면, 그 점은 core point 가 된다.
- border point : 어떤 점이 core point 는 아니지만, core point 의 반경 (Eps) 안에 놓이는 경우이다. (중심은 아니지만 군집에 속하는 점, 클러스터의 경계에 위치)
- noise point : core 도 border 도 아닌 점으로, 어떤 클러스터에도 속하지 않는다.

Border Point (P2)

Noise Point (P4)

step 별로 정리해보면 다음과 같이 동작한다.

1. 임의의 데이터 포인트를 선택한다.
2. 해당 점의 ϵ 반경 내 이웃 개수를 확인한다.
3. 이웃 개수가 Min_samples 이상이면 core point 로 판단하고 클러스터를 생성한다.

4. core point 와 density reachable 한 점들을 계속 확장하여 클러스터를 형성한다.
5. 더 이상 확장할 수 없으면 새로운 점에서 다시 시작한다.
6. 어떤 클러스터에도 포함되지 않은 점들은 noise 로 분류한다.

실습 예제

이번 실습에서는 DBSCAN 을 사용하여 데이터의 밀집 영역을 기반으로 클러스터를 형성하는 과정을 살펴본다.

DBSCAN 은 K-means 와 달리 클러스터 개수를 사전에 지정할 필요가 없으며, 최소 데이터 개수를 기반으로 밀도가 낮은 포인트를 noise(이상치)로 구분할 수 있다는 특징을 가진다.

이번 실습에서는 (K-means 실습에서도 사용했던) 붓꽃(Iris) 데이터의 feature 를 사용해 DBSCAN 알고리즘이 데이터의 밀도 구조를 기반으로 클러스터를 형성하는 방식을 실제로 확인해보려고 한다.

바로 iris dataframe 생성해보자.

```
from sklearn import datasets
import pandas as pd

iris = datasets.load_iris()
labels = pd.DataFrame(iris.target)
labels.columns=['labels']
data = pd.DataFrame(iris.data)
data.columns=['Sepal length', 'Sepal width', 'Petal length', 'Petal width']
data = pd.concat([data, labels], axis=1)
feature = data[ ['Sepal length', 'Sepal width', 'Petal length', 'Petal width']]
feature.head()

{"summary": "{\n  \"name\": \"feature\",\n  \"rows\": 150,\n  \"fields\": [\n    {\n      \"column\": \"Sepal length\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.8280661279778629,\n        \"min\": 4.3,\n        \"max\": 7.9,\n        \"num_unique_values\": 35,\n        \"samples\": [\n          6.2,\n          4.5,\n          5.6\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Sepal width\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.435866284936698,\n        \"min\": 2.0,\n        \"max\": 4.4,\n        \"num_unique_values\": 23,\n        \"samples\": [\n          2.3,\n          4.0,\n          3.5\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Petal length\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1.7652982332594667,\n        \"min\": 1.0,\n        \"max\": 6.9,\n        \"num_unique_values\": 43,\n        \"samples\": [\n          6.7,\n          6.7,\n          6.7\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  }\n}
```

```

3.8,\n          3.7\n          ],\n          \"semantic_type\": \"\", \n          \"description\": \"\" \n          },\n          {\n          \"column\": \n          \"Petal width\", \n          \"properties\": {\n          \"dtype\": \n          \"number\", \n          \"std\": 0.7622376689603465, \n          \"min\": \n          0.1, \n          \"max\": 2.5, \n          \"num_unique_values\": 22, \n          \"samples\": [\n          0.2, \n          1.2, \n          1.3 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n          } \n          } \n          ], \"type\": \"dataframe\", \"variable_name\": \"feature\"}

```

```

from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt
import seaborn as sns

# create model and prediction
model = DBSCAN(eps=0.5,min_samples=5)
predict = pd.DataFrame(model.fit_predict(feature))
predict.columns=['predict']

# concatenate labels to df as a new column
r = pd.concat([feature,predict],axis=1)

print(r)

```

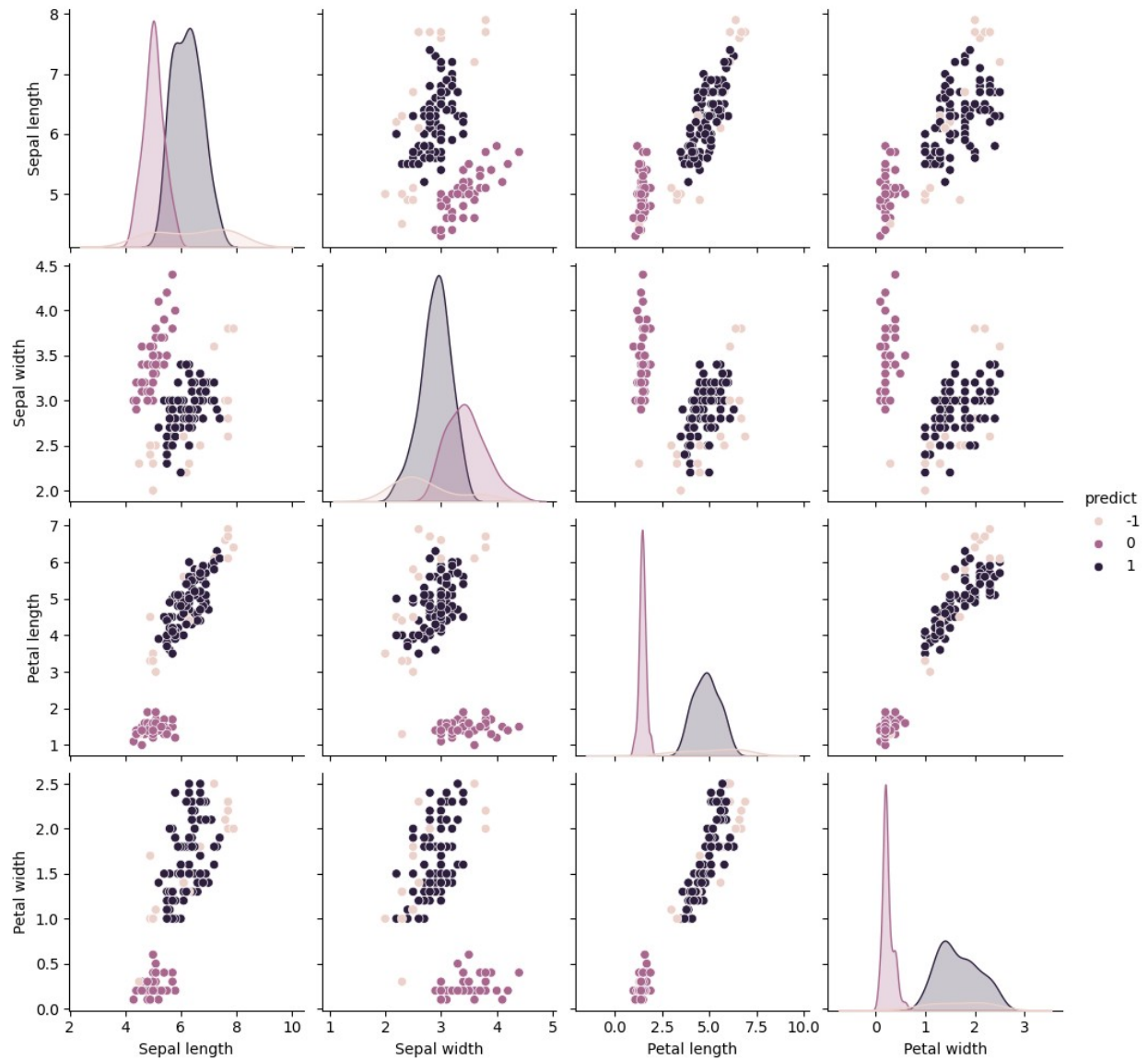
	Sepal length	Sepal width	Petal length	Petal width	predict
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
..	...	...	...	...	...
145	6.7	3.0	5.2	2.3	1
146	6.3	2.5	5.0	1.9	1
147	6.5	3.0	5.2	2.0	1
148	6.2	3.4	5.4	2.3	1
149	5.9	3.0	5.1	1.8	1

[150 rows x 5 columns]

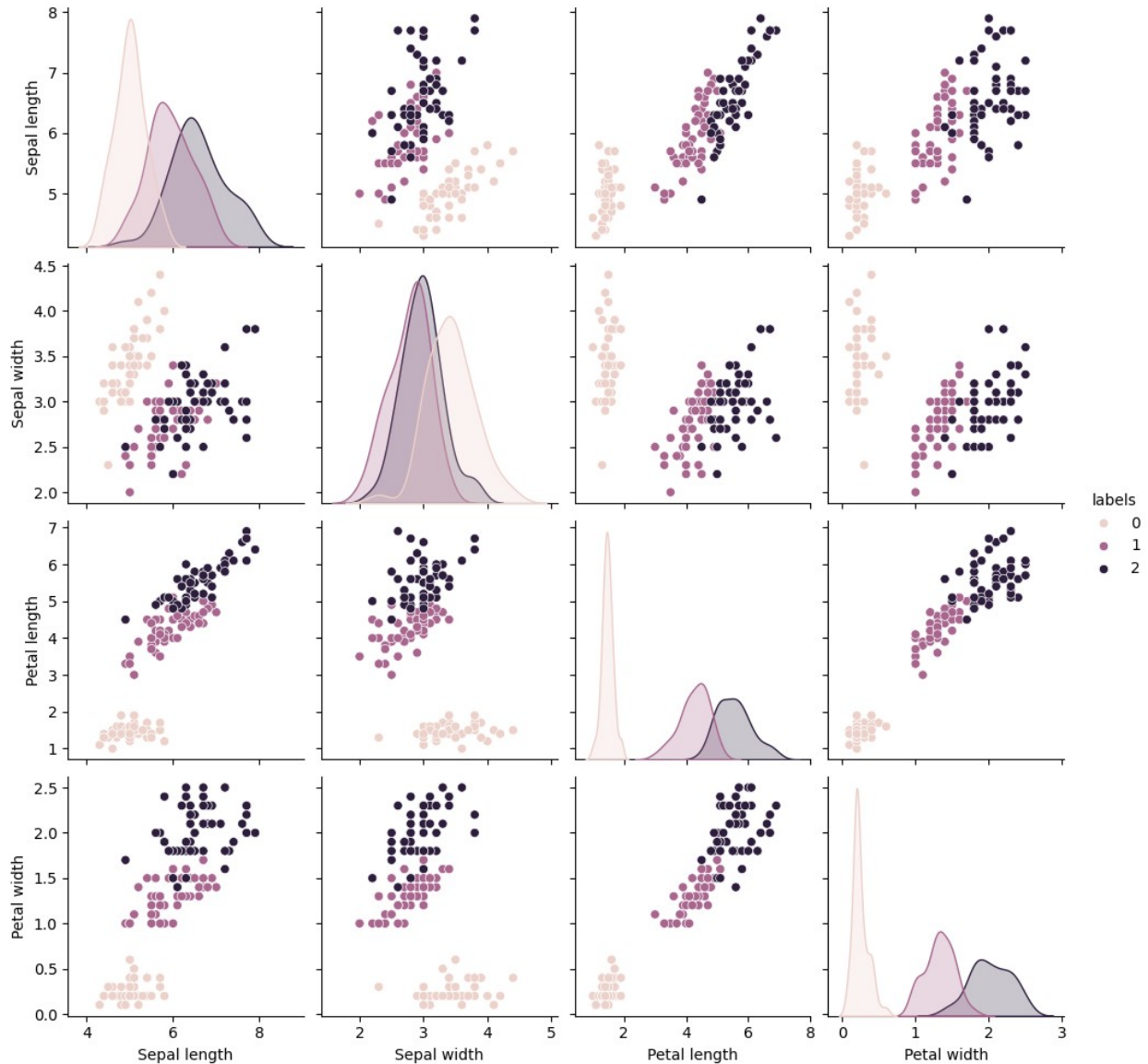
```

#sns 라이브러리의 pairplot 으로 표현
sns.pairplot(r,hue='predict')
plt.show()

```



```
#sns 라이브러리의 pairplot 으로 표현
sns.pairplot(data, hue='labels')
plt.show()
```



3. 차원 축소 (Dimensionality Reduction)

지금까지 살펴본 K-means, Mean-shift, DBSCAN 알고리즘은 데이터를 유사한 그룹으로 나누는 군집화(clustering) 방법이었다.

하지만 실제 데이터 분석에서는 다음과 같은 문제가 자주 발생한다.

- 데이터의 feature 개수가 매우 많은 경우
- 변수 간 상관관계가 강한 경우
- 고차원 데이터로 인해 분석 및 시각화가 어려운 경우

이러한 요인들로 데이터의 구조를 이해하기 어려워지고 분석 비용도 증가할 때 문제를 해소하기 위해 사용하는 대표적인 방법이 바로 차원 축소이다.

대표적인 차원 축소 방법에는 PCA, t-SNE, UMAP 같은 알고리즘들이 존재하지만, 우선 가장 기본적으로 널리 사용되는 PCA에 대해 자세히 학습해보자.

3-1. 주성분 분석, PCA (Principal component analysis)

PCA는 고차원 데이터를 더 낮은 차원의 공간으로 변환하는 대표적인 차원 축소 알고리즘으로, 변수들을 선형 결합(linear combination)하여 주성분(Principal Components)이라는 새로운 변수를 생성한다.

이때 생성되는 주성분은 다음과 같은 특징을 가진다.

- 서로 직교(orthogonal)하며 상관관계가 없음
- 데이터의 분산(variance)이 최대가 되는 방향을 찾음

직관

2차원 데이터가 다음과 같이 분포되어 있다고 가정해보자.

```
feature1 → x 축  
feature2 → y 축
```

데이터가 대각선 방향으로 퍼져 있다면, x 축과 y 축을 그대로 사용하는 것보다 데이터가 퍼져있는 방향으로 새로운 축을 만드는 것이 데이터의 구조를 더 잘 설명할 수 있다.

PCA는 바로 이 데이터 분산이 가장 큰 방향을 찾아 새로운 좌표축으로 사용하는 방법이다.

```
PC1 : 데이터 분산이 가장 큰 방향  
PC2 : PC1 과 직교하면서 두 번째로 큰 분산 방향
```

이렇게 PC1, PC2를 이용하면 고차원 데이터를 더 적은 차원으로 표현할 수 있다.

알고리즘

PCA는 다음과 같은 절차로 수행된다.

1. 평균을 빼고 표준편차로 나눔으로써, 데이터의 평균을 0으로 맞추어 표준화한다 (mean centering)

- 표준화된 데이터를 이용하여 feature 간 관계를 나타내는 공분산 행렬(C, covariance matrix) 계산한다. 공분산 행렬은 변수들이 서로 어떤 방향으로 함께 변하는지를 나타낸다.
- 공분산 행렬의 고유값(eigenvalue)과 고유벡터(eigenvector) 계산한다. 이때 고유벡터는 데이터가 가장 크게 분산되는 방향을 나타낸다.
- 계산된 고유벡터들을 하나의 행렬로 구성하여 데이터에 적용한다. 이 과정에서 데이터는 새로운 좌표축으로 회전(rotation)되고 스케일이 조정된다.
- 고유값이 큰 순서대로 주성분을 정렬한다. 가장 큰 고유값을 가지는 고유벡터가 첫 번째 주성분(PC1)이며, 이는 데이터의 분산을 가장 많이 설명하는 방향이다.
- 상위 k 개의 주성분을 선택하고 데이터를 해당 축으로 투영한다. 이러한 방식으로 중요한 정보는 유지하면서 데이터의 차원을 줄일 수 있다. 보통 $k < n$ 이다. (n은 시작 차원 크기, 원본 특징 개수)

※ PCA는 선형대수학과 밀접하게 관련된 알고리즘이므로, 수식적인 유도나 이론적 배경을 보다 엄밀히 이해하고 싶다면 관련 서적이나 [참고 자료](#)를 통해 추가로 학습해보기를 권한다.

알고리즘의 장단점:

- 장점
 - 데이터 차원을 효과적으로 축소 가능
 - 데이터 시각화에 유용
 - 변수 간 상관관계를 제거
 - 계산 비용 감소
- 단점
 - 선형 관계만 모델링 가능
 - 변수 스케일에 민감
 - 해석이 어려울 수 있음
 - 이상치(outlier)에 영향을 받을 수 있음

Clustering 과 PCA

PCA와 clustering은 모두 비지도 학습에 속하지만 서로 다른 목적을 가진다.

구분	PCA	Clustering
목적	차원 축소	데이터 그룹화
결과	새로운 feature 생성	cluster label 생성

구분	PCA	Clustering
예시	PCA	K-means, DBSCAN

실습 예제

이번 실습에서는 PCA 를 이용하여 고차원 데이터를 저차원으로 변환하는 과정을 확인해본다. 마찬가지로 붓꽃 데이터를 활용한다.

이번에는 총 4 개의 feature (sepal length, sepal width, petal length, petal width) 를 모두 사용하여 PCA 를 통해 4 차원 데이터 → 2 차원 데이터로 변환하고, 데이터가 어떻게 분포하는지 시각화해보자.

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import pandas as pd
import matplotlib.pyplot as plt

iris = load_iris()
# print(iris.keys())
print(f"feature ({len(iris.feature_names)}) :", iris.feature_names)
print(f" target ({len(iris.target_names)}) :", iris.target_names)

feature (4) : ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
target (3) : ['setosa' 'versicolor' 'virginica']

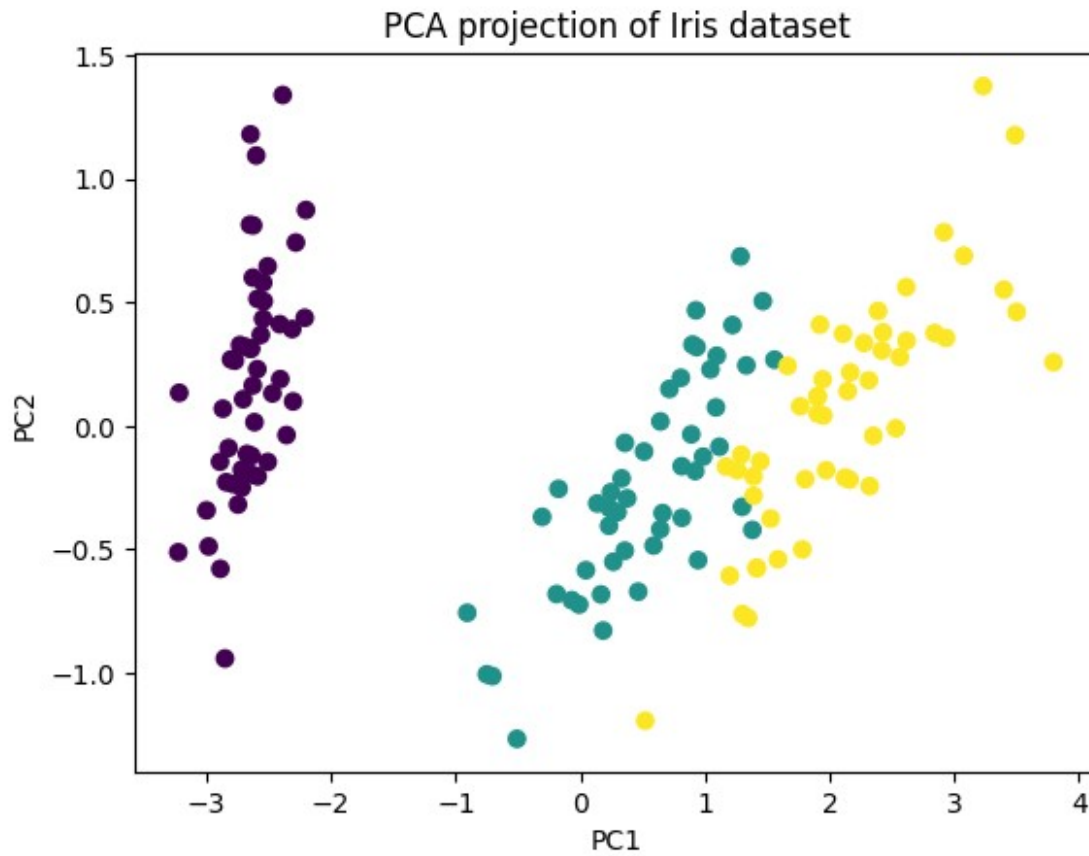
X = iris.data
y = iris.target

print(X.size)
print(y.size)

600
150

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

plt.scatter(X_pca[:,0], X_pca[:,1], c=y)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA projection of Iris dataset")
plt.show()
```

원래 4 차원 feature 공간에 존재하는 데이터가 2 차원 공간으로 변환된 모습을 확인할 수 있다!

이렇게 변환된 데이터는

- 데이터 구조를 시각적으로 이해하기 쉽고
- 이후 clustering 알고리즘을 적용하기에도 유리하다.